

Intelligent Guide v1 — Operations Runbook

On-call playbook for the v1 onboarding journey

Joe Pruskowski

2026-04-25

Contents

Why this doc exists	1
1. Incident response	1
2. Tuning SystemConfig safely	3
3. Reading the dashboard	5
4. Escalation	5
5. Common operations cookbook	5
6. References	9

Why this doc exists

The Intelligent Guide v1 is the seven-step onboarding journey (Hook → Curriculum → Specialize) plus the dashboards and rewards behind it. This document is what you read when something on the dashboard looks wrong, or when you want to safely tune one of the SystemConfig knobs.

It assumes you already know the *what* (Intelligent_Guide_Requirements.md) and *how* (Intelligent_Guide_Implementation_Plan.md). This is the *what to do when something is on fire* doc.

Companion docs:

- Intelligent_Guide_Requirements.md — the spec
- Intelligent_Guide_Implementation_Plan.md — sprints + checklists
- V1_Acceptance.md — end-to-end QA script (run before every release)
- Guide_Operations.md (this doc) — incident response + tuning

1. Incident response

1.1 The signal

The single source of truth is the **/admin/guide-metrics** dashboard:

- **North Star** — % of users (signed up ≥ 30 days ago) whose bot played a tournament within 30 days. Healthy $\geq 40\%$.
- **Funnel** — completion count for each of the 7 steps. Healthy: drop-off $< 30\%$ per step.
- **Signup split** — credential vs OAuth share over the last 30 days.
- **Trend line** — Day / Week / Month rollup; pivot the granularity dropdown to find the right slice.

Footer reads “Excluding N test users” — internal accounts (any email matching `metrics.internalEmailDomains`) are filtered out so dashboard noise doesn’t drown out real users.

1.2 Triage table

When a metric drops, work top-to-bottom. Likely cause first, easy diagnostic second, first remediation third. Do not skip diagnostics — most “the dashboard is wrong” reports are actually a single test-user account that wasn’t flagged.

Metric drops	Likely cause	Diagnostic	First remediation
North Star plummets	Big batch of test users not flagged → dashboard denominator inflated	<code>um testuser --audit</code> (lists drift between <code>metrics.internalEmailDomains</code> and <code>User.isTestUser</code>)	<code>um testuser --apply</code> to flag everyone matching the domain list
North Star drifts down slowly	Real cohort regression — bots not being entered into tournaments	Spot-check a few cohort users via <code>um status <user></code> (look at <code>journeyProgress.completedSteps</code>) cross-reference with <code>/admin/tournaments</code>	Check tournament sweep cron on tournament service; investigate Discovery <code>firstRealTournamentWin</code> reward in logs
Step 1 count drops	PvAI completion path broken	<code>docker compose logs backend \ grep "Journey step completed"</code> should show step 1 firing on PvAI game end	Check back-end/src/routes/games.js — the <code>completeStep(userId, 1)</code> call after a successful game record
Step 2 count drops	Demo-table watch credit broken — usually a TableGC change or socket listener regression	Repro: sign up a fresh user, navigate to a demo table, wait for the bot game to finish. Watch backend logs for Journey step completed { stepIndex: 2 }	Confirm <code>tableGcService.sweepDemos</code> isn't reaping mid-watch (check <code>guide.demo.ttlMinutes</code>); confirm socket <code>table:watch</code> listener fires
Steps 3-5 count drops	Quick Bot wizard / Quick Train / Spar endpoint regression	<code>um status <user></code> for any new user; check <code>/api/v1/bots/quick</code> , <code>/api/v1/bots/:id/train-quick</code> , <code>/api/v1/bot-games/practice</code> POST handlers	Check the relevant handler isn't crashing — they all call <code>completeStep</code> directly after the side-effect succeeds
Step 6 count drops	Cup clone or participant <code>participant:joined</code> publish broken	<code>curl -X POST /api/tournaments/curriculum/cup/clone</code> as a test user; confirm a participant row appears	Check tournament/src/services/curriculumCupService and the bridge subscriber back-end/src/lib/tournamentBridge.js
Step 7 count drops	Cup completion not firing OR <code>tournament:completed</code> publish stuck	Watch tournament logs for the cup running; if it completes but step 7 doesn't fire, the bridge is the suspect	Check tournament-Bridge.coachingCard subscription — same path emits step 7
Signup split flips abruptly	OAuth provider down OR credential signup broken	Check <code>/api/auth/sign-in/email</code> and <code>/api/auth/sign-in/social/*</code> directly	If OAuth is the culprit, check Better Auth provider env vars; if credential — check the <code>User.create</code> path is committing
Funnel flat across the board	<code>guide.v1.enabled</code> was flipped off	<code>curl /api/v1/admin/guide-config \ jq '. "guide.v1.enabled"'</code>	Flip back on via the admin SystemConfig panel; in-flight credits during the off window are gone (not recoverable)
Test-user count trending up	Internal accounts not being flagged correctly	<code>um testuser --audit</code>	Add the missing domain to <code>metrics.internalEmailDomains</code> via admin UI; then <code>um testuser --apply</code>

1.3 The “everything’s broken” case

If multiple metrics drop at once — first check the **release flag**:

```
curl -s http://localhost:3000/api/v1/admin/guide-config \
-H "Authorization: Bearer <admin-token>" \
| jq '.<guide.v1.enabled>'
```

If that’s false, journey credits and discovery rewards are silently no-op. Flip it back on via the admin UI (/admin → Intelligent Guide v1 panel) — the change takes effect immediately, no restart needed.

1.4 Backfill after downtime

If the metrics cron didn’t run (deploy / outage / DB downtime), run:

```
docker compose exec backend node src/scripts/backfillMetrics.js [--days N] [--dry-run]
```

The script walks past UTC days and rewrites North Star + signup-split rows. **Funnel + test-user-count are NOT backfilled** — those metrics have no historical state, so old values would be a flat replay of today.

--dry-run prints what would be written without touching the DB. --days 14 re-walks the last 14 days only. Idempotent — safe to re-run for the same range.

2. Tuning SystemConfig safely

The admin UI (/admin → Intelligent Guide v1 panel) exposes 13 keys. All changes take effect immediately for new triggers — already-granted rewards are not retroactively adjusted.

Key	Type	Default	Safe range	Reversibility
guide.v1.enabled	boolean	true	true/false	Flip to off loses any journey credits / discovery rewards that fire during the off window. Other gameplay (games, bots, tournaments) is unaffected.
guide.rewards.hookComplete	integer	20	0–100	Reversible. New users at step 2 get the new value. Already-granted rewards stay.
guide.rewards.curriculumComplete	integer	50	0–200	Reversible. New users at step 7 get the new value.
guide.rewards.discoveryFirstSpecializeAction	integer	10	0–100	Reversible. Surface ships in v1 with no caller — no observable effect until v1.1.
guide.rewards.discoveryFirstRealTournament	integer	25	0–100	Reversible. Granted on first tournament win that is <i>not</i> a curriculum cup.

Key	Type	Default	Safe range	Reversibility
<code>guide.rewards.discoveryIntegerFirstNonDefaultAlgorithm</code>	integer	10	0–100	Reversible. Granted when a user trains a bot with a non-copy-rusty algorithm (qLearning, dqn, etc.).
<code>guide.rewards.discoveryIntegerFirstTemplateClone</code>	integer	10	0–100	Reversible. Surface ships in v1 with no caller. v1.1 wires user-facing template clone.
<code>guide.quickBot.defaultTier</code>	enum	novice	novice / intermediate / advanced / master	Reversible. Wizard-created bots get this tier going forward.
<code>guide.quickBot.firstTrainingTier</code>	enum	intermediate	same enum	Reversible. After Quick Train, the bot flips to this tier.
<code>guide.cup.sizeEntrant</code>	integer (read-only)	4	n/a	Read-only in v1 — admin UI rejects writes. The cup spawn logic hardcodes the 4-bot slot mix in <code>tournament/src/config/curriculumCup</code> changing the <code>SystemConfig</code> value alone would have no behavioral effect. v1.1 wires it.
<code>guide.cup.retentionDays</code>	integer	30	1–365	Reversible. The tournament sweep deletes COMPLETED cups older than this.
<code>guide.demo.ttlMinutes</code>	integer	60	5–1440	Reversible. Demo tables get reaped after this many idle minutes. Don't drop below 5 — short-running matches will get killed mid-game.
<code>metrics.internalEmailDomain</code>	string array	[]	any	Reversible, but: removing a domain doesn't un-flag accounts that were already flagged (run <code>um testuser --audit</code> to see the drift, then <code>--apply</code> to reconcile).

2.1 Safe-tuning pattern

Before changing a reward value:

1. Note the current value.

2. Make the change in the admin UI.
3. Watch the funnel + signup-split panels for the next 24 hours.
4. If something drops, flip back. The rollback is instant.

For the release flag specifically, prefer **off** → **on** to “patch fast and ship.” Off → on is reversible; off-window credits are not.

3. Reading the dashboard

3.1 Healthy

- North Star $\geq 40\%$ on the 7-day moving average.
- Funnel: drop-off $< 30\%$ per step (i.e., step N+1 retains $>70\%$ of step N).
- Signup split: stable ratio. A sudden flip is a signal.
- Test-user footer: stable count, growing only when intentional internal signups happen.

3.2 Warning (investigate same day)

- North Star drops 5+ points week-over-week.
- One funnel step drops 50%+ overnight.
- Signup split flips by >30 points overnight.
- Test-user count grows by >5 in one day with no known internal onboarding.

3.3 Fire (investigate immediately)

- North Star at 0% (denominator collapsed → flag audit issue, or genuine outage).
- All funnel steps at 0 (release flag flipped off).
- Signup split is 100/0 (one auth path is broken).
- Dashboard returns 500 or hangs (admin endpoint or metrics cron broken).

3.4 Cohort granularity

The trend chart on the North Star panel has a Day / Week / Month picker (default Week). Use:

- **Day** — for high-volume periods or to spot a single bad-day regression.
- **Week** — default; smooths weekday/weekend noise.
- **Month** — for long-range comparisons; daily buckets get too sparse.

The bucketing is client-side only — the underlying snapshot data doesn’t change.

4. Escalation

For Sprint 1 (the only sprint at time of writing), **everything routes to Joe**. The goal is to grow this list as the team grows:

- **First responder** — Joe (any time)
- **Backup** — n/a yet; document who replaces Joe when the on-call schedule exists
- **Tournament service issues** — Joe (the bridge subscriber + cup logic both live in tournament/)
- **Auth issues** — Joe (Better Auth + OAuth provider env vars)

The pattern future on-call docs should follow: incident type → first responder → escalation step 1 → step 2.

5. Common operations cookbook

Copy-paste recipes for routine ops. Each one has been smoke-tested at least once on local dev.

5.1 Onboard a new internal admin

```
um create <name> --admin
```

Then verify the auto-flag fired (the email-domain rule should have set `isTestUser=true` if the email matches `metrics.internalEmailDomains`):

```
um testuser <name>
```

If it didn't auto-flag (e.g., the user's email is on a personal domain), force the flag:

```
um testuser <name> --on
```

5.2 Audit + reconcile test-user drift

```
um testuser --audit           # list users whose flag disagrees with the domain list
um testuser --apply           # reconcile: flag everyone matching, unflag everyone not matching
```

The audit is read-only. `--apply` is the only command that writes; review the audit output first.

5.3 Backfill the dashboard after a long downtime

```
docker compose exec backend node src/scripts/backfillMetrics.js
```

Optional: limit the lookback with `--days 30`. Use `--dry-run` to preview without writing. The script is idempotent so re-running for the same range is safe.

5.4 Disable the guide for a hotfix

1. Open `/admin`, scroll to “Intelligent Guide v1” panel.
2. Uncheck **guide.v1.enabled**.
3. The browser confirm dialog explains the cost — accept.
4. Click Save.

Effect: journey credits and discovery rewards become no-ops within milliseconds. Other gameplay continues normally. The metrics cron still runs and dashboards still update.

5.5 Re-enable the guide

1. Open `/admin` → Intelligent Guide v1 panel.
2. Check **guide.v1.enabled**.
3. Click Save.

No confirm dialog when re-enabling. Effect is immediate. The next user action that would fire a step credit will go through.

5.6 Inspect a single user's journey state

```
um status <user>           # phase + isTestUser + grants + TC
um rewards show <user>      # discovery reward grants
```

If a user reports “I should have gotten the +50 TC for completing the curriculum but didn't” — start here. Look for step 7 in `completedSteps`; if present, the reward fired. If TC didn't increment, check the `User.creditsTc` write path in `journeyService._handleCurriculumComplete`.

5.7 Replay a failed reward

There is intentionally no “replay” command. The audit trail is the logger output:

```
docker compose logs backend | grep "Hook complete\|Curriculum complete"
```

If the reward log line is missing, the trigger never fired (root cause is upstream). If the log line is there but the user reports no TC, something failed in the DB write — re-run the audit + manually grant via Prisma if necessary, but log the manual grant.

5.8 Service Worker kill switch

The landing app's Service Worker (Phase 20, app-shell caching) reads two SystemConfig keys on each `install / activate` and on tab visibility-change, via the public `GET /api/v1/config/sw` endpoint:

- **sw.enabled** (*bool, default true*) — the kill switch. Flip to false and every SW in the wild self-unregisters and clears all caches on its next check-in (within ~30s, bounded by the response's `Cache-Control: max-age=30`).
- **sw.version** (*int, default 1*) — bump to invalidate the SW's precache without unregistering the worker. Use this when a cached asset URL is wrong or poisoned but the SW logic itself is fine.

When to flip the kill switch: - A bad SW deploy is serving stale or corrupted assets to users and you can't roll forward fast. - Auth or `/api/*` requests are mysteriously hitting cached responses (the SW should never cache `/api/*` — if you suspect it is, kill it while you debug). - A user reports "I see an old version of the app even after a hard reload" and you need a fleet-wide eject.

To kill the SW (revert when fixed):

```
# Set sw.enabled = false
docker compose exec -T backend node --experimental-transform-types --no-warnings -e "
import('./src/lib/db.js').then(async ({ default: db }) => {
  await db.systemConfig.upsert({
    where: { key: 'sw.enabled' },
    create: { key: 'sw.enabled', value: false },
    update: { value: false },
  })
  console.log('sw.enabled = false - SWs will self-unregister on next check-in')
  await db.$disconnect()
})
"
```

Re-enable by upserting value: `true` (or deleting the row, since the default is true). Bump cache version with key: `'sw.version', value: <currentVersion + 1>`.

Verification: open DevTools → Application → Service Workers on a tab you know had the SW registered; within 30s of flipping `sw.enabled=false`, the SW should disappear and `caches.keys()` in the console should return `[]`.

5.9 Research-Log community publishing toggle

The Profile-page Training Journal lets users publish their own notes and research-log entries to a shared community lane (see `doc/Research_Log_Plan.md` Sprint 2). The publish surface is gated by a SystemConfig key — when the row is absent the flag is **OFF** and every publish/unpublish endpoint returns 503 `{ error: 'publish_disabled' }` without touching the DB. Flipping is a one-row upsert, no redeploy required.

Default state is OFF. Ship the feature dark, watch metrics + abuse reports for a few days, then flip ON. The frontend modal handles 503 by rendering "Community publishing is currently disabled."

To enable community publishing:

```
docker compose exec -T backend node --experimental-transform-types --no-warnings -e "
import('./src/lib/db.js').then(async ({ default: db }) => {
  await db.systemConfig.upsert({
    where: { key: 'researchLog.publishEnabled' },
    create: { key: 'researchLog.publishEnabled', value: { enabled: true } },
    update: { value: { enabled: true } },
  })
  console.log('researchLog.publishEnabled = true')
  await db.$disconnect()
})
"
```

The `isPublishEnabled` helper accepts any of value: `true`, value: `'true'`, or value: `{ enabled: true }` — pick whichever feels clearest. The `{ enabled: <bool> }` shape leaves room for adjacent sub-flags later without a key rename.

To disable (emergency stop):

```
docker compose exec -T backend node --experimental-transform-types --no-warnings -e "
import('./src/lib/db.js').then(async ({ default: db }) => {
  await db.systemConfig.upsert({
    where: { key: 'researchLog.publishEnabled' },
    create: { key: 'researchLog.publishEnabled', value: { enabled: false } },
    update: { value: { enabled: false } },
  })
  console.log('researchLog.publishEnabled = false')
  await db.\$disconnect()
})
"
```

Disabling does **not** retroactively unpublish anything — existing HelpDoc rows with source = 'community-note' stay in place and remain visible to the Sprint 3 retrieval mixer. To purge the community lane, delete those rows directly:

```
docker compose exec -T backend node --experimental-transform-types --no-warnings -e "
import('./src/lib/db.js').then(async ({ default: db }) => {
  const docs = await db.helpDoc.findMany({ where: { source: 'community-note' }, select: { id: true } })
  ↪ })
  if (docs.length === 0) { console.log('nothing to purge'); return await db.\$disconnect() }
  await db.helpChunk.deleteMany({ where: { docId: { in: docs.map(d => d.id) } } })
  await db.helpDoc.deleteMany({ where: { id: { in: docs.map(d => d.id) } } })
  console.log('purged ' + docs.length + ' community docs + their chunks')
  await db.\$disconnect()
})
"
```

Per-user rate limit: 50 publishes/user/rolling-7-days, enforced in backend/src/middleware/researchPublishRateLimit.js. Unpublish is not rate-limited. The 429 response carries { error: 'rate_limited', scope: 'week', retryAfter, limit }.

5.10 Editing an existing Help-Corpus doc in staging or production

The Help-Corpus seeder (seedCorpus() in backend/src/services/help/corpusSeeder.js) runs on every backend boot and is **additive-only by design**:

- **New slug** in /doc/Help_Corpus/*.md → inserted + chunked + embedded automatically on the next boot. No manual step.
- **Edit to an existing slug** → **NOT** picked up on boot. The seeder checks “does this slug already exist?” and skips if yes. The live row keeps the prior body.
- **Deleted file** → the corresponding row stays in help_docs. The seeder never deletes.

This is intentional: the DB is the source of truth after the first seed, so the admin editor at /admin/help can edit a doc without the next boot stomping the edit. The trade-off is that *intended* git-side edits also don’t propagate automatically.

Two ways to propagate an edit to staging or prod:

Option A — the admin editor (preferred for one-off edits) 1. Sign in as a HELP_ADMIN. 2. Go to /admin/help → click the doc → edit in the rich-text editor. 3. Save. The row is updated, chunks regenerate, embeddings refresh. 4. The git file is now out of sync with the DB — *that’s expected*. The admin editor is the source of truth after the first seed. Bring the git copy back into sync via `um help-export` if you want.

Option B — delete + reseed (preferred for git-driven bulk edits)

Use this when you’ve made the edit on dev and merged through the normal dev → staging → main pipeline, and you want the deployed backend to pick up the new file content.

On the target environment (run from a host with fly access):

Staging

```
fly ssh console -a xo-backend-staging -C "node --experimental-transform-types --no-warnings -e \"
```



```
import('/app/backend/src/lib/db.js').then(async ({ default: db }) => {
  const slug = 'SLUG-HERE';
  const doc = await db.helpDoc.findUnique({ where: { slug } });
  if (doc) {
    await db.helpChunk.deleteMany({ where: { docId: doc.id } });
    await db.helpDoc.delete({ where: { id: doc.id } });
    console.log('Deleted', slug);
  }
  const { seedCorpus } = await import('/app/backend/src/services/help/corpusSeeder.js');
  console.log(await seedCorpus());
  await db.\\\ disconnect();
})
\''
```

```
# Production — same command, swap the app name
fly ssh console -a xo-backend-prod -C "..."
```

Replace SLUG-HERE with the slug of the doc you edited (e.g. research-notes-and-journal). The brief window between delete and seedCorpus() is sub-second; queries hitting it during that window get fewer chunks but never an error. For a fleet-wide bulk refresh (multiple edits at once), run the script with a list of slugs.

Aliases + retrieval logic propagate normally — they’re code, not data. The change deploys with the next /stage or /promote; nothing extra needed.

Future improvement noted in the Help-System Question Backlog: add a --force flag on the seeder (or wrap delete+reseed in um help-reseed <slug>) so this isn’t tribal knowledge. Until then, this section IS the runbook.

6. References

- **Spec** — /doc/Intelligent_Guide_Requirements.md
- **Implementation plan** — /doc/Intelligent_Guide_Implementation_Plan.md
- **Acceptance script** — /doc/V1_Acceptance.md
- **Sprint 6 kickoff** — /doc/Sprint6_Kickoff.md
- **Backfill script** — backend/src/scripts/backfillMetrics.js
- **Snapshot service** — backend/src/services/metricsSnapshotService.js
- **Admin endpoints** — backend/src/routes/admin.js (/guide-metrics, /guide-config)
- **Admin UI** — landing/src/pages/admin/{GuideMetricsPage,GuideConfigPanel}.jsx